API REFERENCE

Cyber-Physical Smart Building Access Control System — REST API & MQTT Topics

1. Conventions

Base URL: http://<host>:8000 in development (uvicorn default). All request/response bodies are JSON. Authenticated endpoints require an Authorization: Bearer <token> header, obtained from POST /api/auth/login.

Auth level	Meaning
public	No token required
user	Any valid token (admin or instructor role)
admin	Token must carry the admin role — non-admins get HTTP 403

Errors follow FastAPI's default shape: {"detail": "<message>"} with a 4xx/5xx status code. 401 means missing/invalid/expired token, 403 means valid token but insufficient role, 404 means the resource doesn't exist, 409 means a conflict (e.g. duplicate card UID), 429 means rate-limited.

2. Authentication

POST /api/auth/login *[public]*

Request: { "email": string, "password": string }

Response 200: { "access_token": string, "token_type": "bearer" }. The token is a JWT with claims { sub: email, role: "admin"|"instructor", exp }, valid for JWT_EXPIRE_MINUTES (default 60).

Response 401: wrong email/password. Response 429: this account has failed LOGIN_MAX_ATTEMPTS (default 5) times in the last LOGIN_LOCKOUT_SECONDS (default 300s) — see the login response's detail message for the remaining lockout time.

POST /api/auth/refresh *[user]*

Reissues a token for the current session with a fresh expiry. No body. Note: this is not a refresh-token flow with revocation — see Known Limitations in the System Test Report.

3. Users

GET /api/users *[admin]*

Returns every user: [{ user_id, name, email, role }].

POST /api/users *[admin]*

Request: { name, email, role: "admin"|"instructor", password }. Response 201: the created user (password is hashed with bcrypt before storage, never returned).

4. Credentials

GET /api/credentials *[admin]*

Returns every credential: [{ credential_id, user_id, card_uid_masked, active, issued_at }]. card_uid_masked looks like "****BEEF" — the real UID is Fernet-encrypted at rest (Phase 5) and is never returned in full over the API.

POST /api/credentials *[admin]*

Request: { user_id?, card_uid?, fp_template_hash? }. The plaintext card_uid in the request is encrypted and indexed before storage; it is never stored or logged in plaintext. Response 201: the created credential (masked). Response 409: a credential with this card UID already exists (checked via the blind index, not by decrypting every row).

DELETE /api/credentials/{credential_id} *[admin]*

Revokes (soft-deletes: active=false) rather than hard-deleting, since access_events keeps a foreign key to this row for audit purposes. Response 204, no body.

5. Doors

GET /api/doors *[user]*

Returns every door: [{ door_id, code, name, building, fail_mode, online, locked, last_seen }]. online/locked/last_seen are updated by the MQTT ingestion service (Phase 3) and the staleness watchdog (Phase 7) — they reflect live status, not configuration.

GET /api/doors/{door_id} *[user]*

Single door, same shape as above. 404 if it doesn't exist.

GET /api/doors/{door_id}/logs *[user]*

Returns up to 200 most recent access events for this door: [{ event_id, door_id, credential_id, event_time, method, result }], newest first. method is one of card, card+fingerprint, override, rex; result is granted or denied (override logs use sent/queued_no_broker in the result field instead, since an override isn't a credential check).

POST /api/doors/{door_id}/override *[admin]*

Request: { "action": "lock"|"unlock" }. Publishes site/{code}/cmd over MQTT (relayed by the Building Gateway if this door is on the RS-485 backbone) and logs the attempt to access_events regardless of delivery. Response: { door_id, action, mqtt_delivered: bool } — mqtt_delivered false means the broker wasn't reachable, not that the command failed after being sent.

6. Schedules

GET /api/schedules *[user]*

Returns every schedule: [{ schedule_id, door_id, day_of_week (0=Monday..6=Sunday), start_time, end_time, course_id }].

POST /api/schedules *[admin]*

Request: { door_id, day_of_week, start_time, end_time, course_id? }. 404 if door_id doesn't exist.

PUT /api/schedules/{schedule_id} *[admin]*

Partial update — any subset of day_of_week/start_time/end_time/course_id. 404 if the schedule doesn't exist.

7. Alerts

GET /api/alerts *[user]*

Optional query param ?resolved=true|false to filter. Returns [{ alert_id, door_id, type, alert_time, resolved }], newest first. type is one of tamper, forced, offline, propped_open, access_denied, auth_failed.

PUT /api/alerts/{alert_id}/resolve *[admin]*

Marks an alert resolved. Returns the updated alert. 404 if it doesn't exist.

8. Misc

GET /health *[public]*

Liveness check: { "status": "ok" }.

GET **/docs** *[public]*

FastAPI's auto-generated interactive API explorer (Swagger UI) — always in sync with the code, not a separate document to maintain.

9. MQTT Topics

The backend and Building Gateway both speak these topics; door node firmware uses them directly (Wi-Fi path) or via the gateway (RS-485 path) — identical either way from the backend's point of view.

Topic	Direction	Payload
site/{code}/status	node/gateway -> backend	"online" or "offline" (plain text, retained)
site/{code}/event	node/gateway -> backend	{ method, result, [card_uid], [queued, queued_for_ms] }
site/{code}/alert	node/gateway -> backend	{ type, [uptime_ms] }
site/{code}/cmd	backend -> node/gateway	{ "cmd": "lock"|"unlock"|"clear_tamper" }
site/{code}/cmd/ack	node/gateway -> backend	{ door_id, cmd, ack: true }

{code} is the door's human-readable code (matches the code column in the Doors table and each node's DOOR_ID firmware constant) — not the numeric door_id primary key.

10. RS-485 Frame Protocol (Gateway <-> Door Node)

Internal to the Building Gateway <-> door node link (Phase 6); the backend never sees this directly. Frame: SOF(0x7E) | ADDR | TYPE | LEN | PAYLOAD (JSON) | CHECKSUM (XOR) | EOF(0x7F), with byte-stuffing for SOF/EOF/ESC bytes inside the payload. TYPE is 0x01 (POLL, gateway to node) or 0x02 (DATA, node to gateway). Full details: door_node_firmware/include/rs485_protocol.h.